FAT CONCEPTS AND ANALYSIS

DATA STRUCTURES IN EACH DATA CATEGORY

Data Category	FAT File System
File System	Boot sector, FSINFO
Content	Clusters, FAT
Metadata	Directory entries, FAT
File Name	Directory entries
Application	N/A

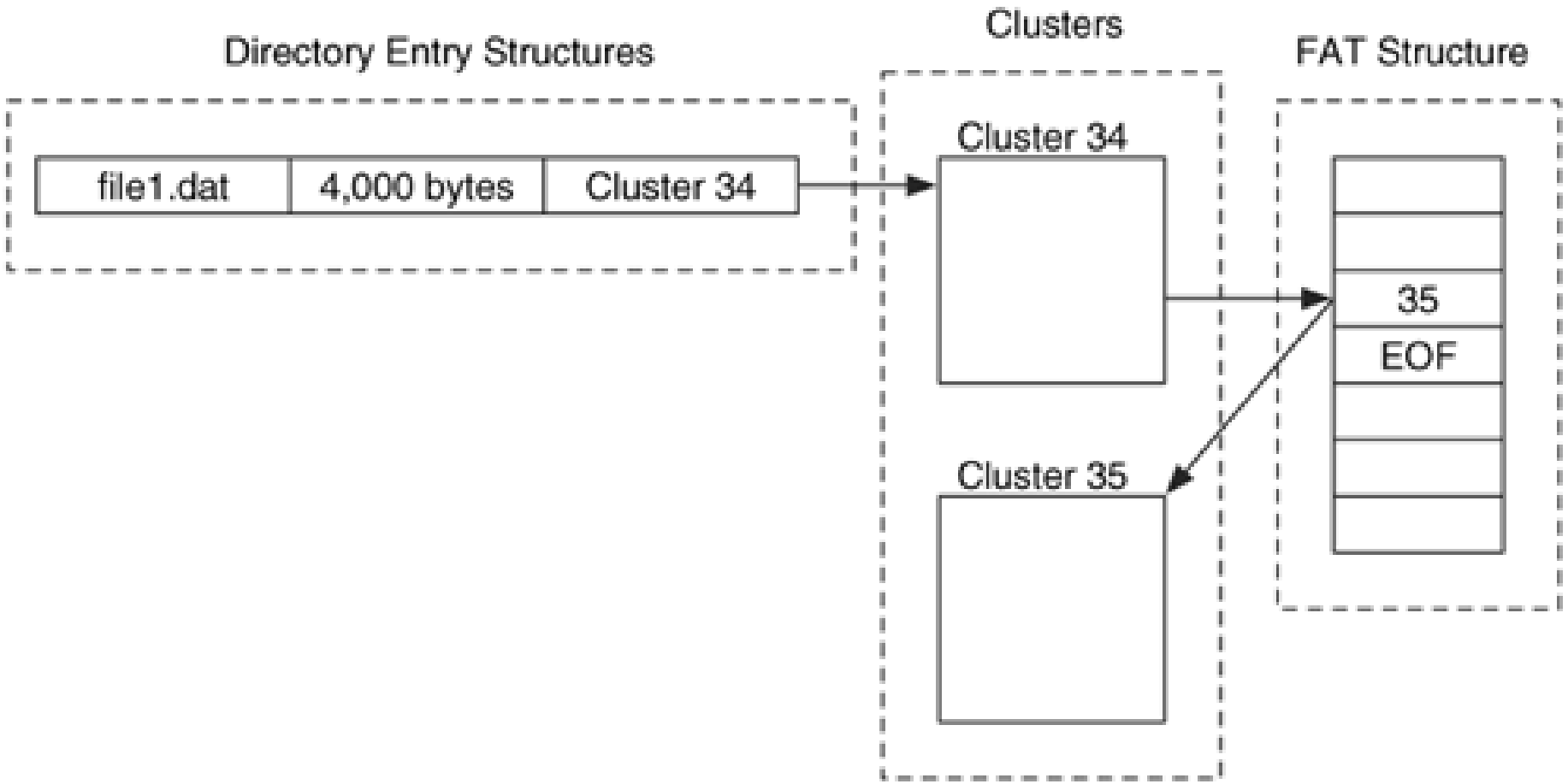
FILE ALLOCATION TABLE (FAT) FILE SYSTEM

- one of the most simple file systems due to its small number of data structure types
- primary file system of the Microsoft DOS and Windows 9x OS.
- supported by all Windows and most Unix OS
- frequently found in compact flash cards for digital cameras and USB "thumb drives.“
- Two important data structures in FAT that belong to multiple data categories:
  - File Allocation Table
  - Directory entries

FILE ALLOCATION TABLE (FAT) FILE SYSTEM

- Each file and directory is allocated a **directory entry** that contains the file name, size, starting address of the file content and other metadata
- File and directory content is stored in data units called clusters.
- FAT structure is used to identify the next cluster in a file, and its allocation status
- 3 different versions based on the size of the entries in the FAT structure: FAT12, FAT16, and FAT32.

RELATIONSHIP BETWEEN THE DIRECTORY ENTRY STRUCTURES, CLUSTERS, AND FAT STRUCTURE



PHYSICAL LAYOUT

- 3 physical sections
  - Reserved area: includes data in the file system category.
  - FAT area: contains the primary and backup FAT structures.
  - Data area: contains the clusters that will be allocated to store file and directory content.

FILE SYSTEM CATEGORY

FILE SYSTEM CATEGORY

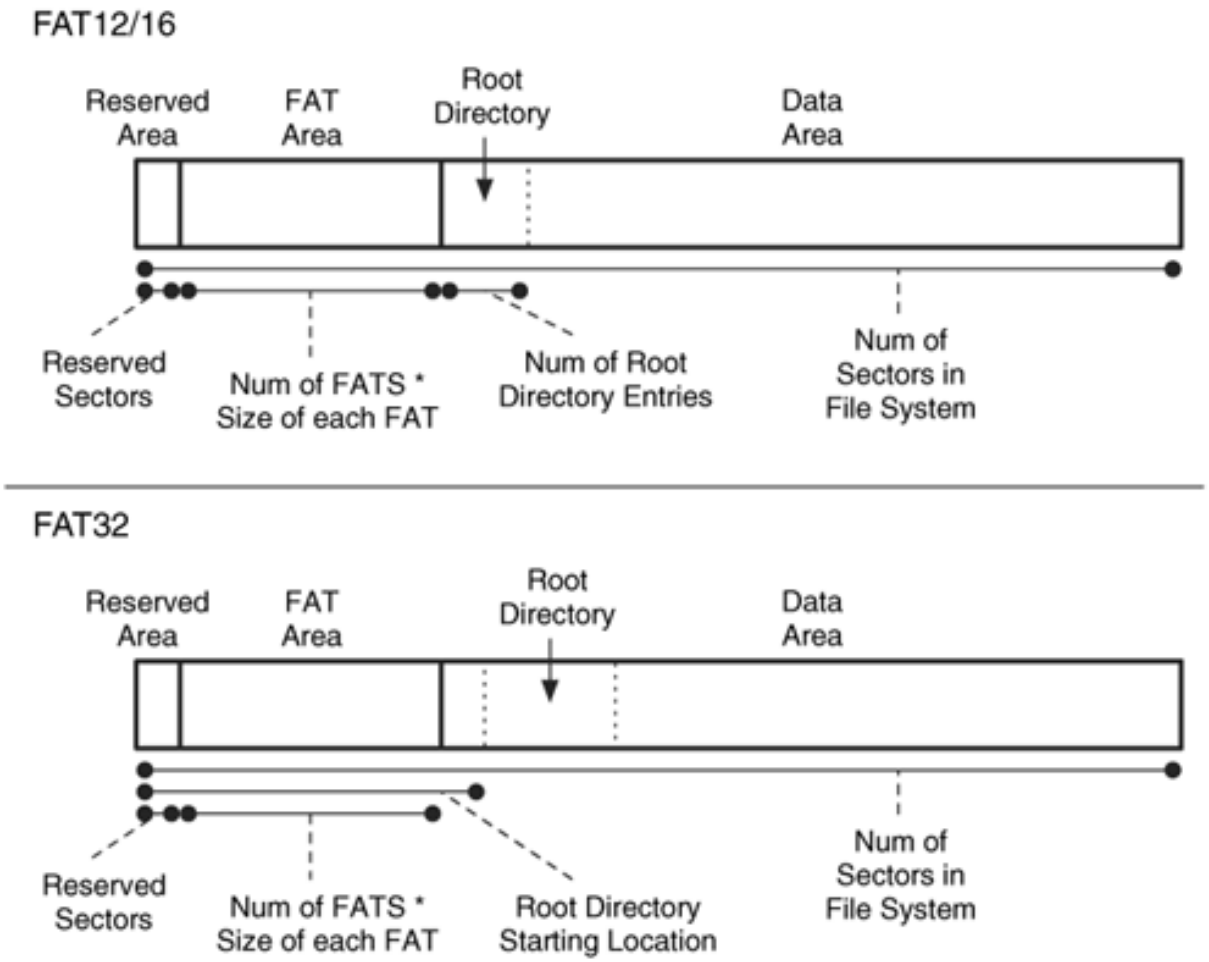
- found in the boot sector data structure located in the first sector of the volume i.e., part of the reserved area.
- FAT type can be determined only by performing calculations from the boot sector data
- FAT32 boot sector contains additional data, including the sector address of a backup copy of the boot sector (always sector 6) and a major and minor version number.
- FAT32 also have an FSINFO data structure that contains information about the location of the next available cluster and the total amount of free clusters.

ESSENTIAL BOOT SECTOR DATA

- location of the three physical layout areas
  - Reserved area:
    - Starts in sector 0 of the file system
    - Size is defined in the boot sector. Only 1 sector in size for FAT12/16
  - FAT area:
    - Size is calculated by multiplying the number of FAT structures by the size of each FAT
    - Both values are given in the boot sector
  - Data area:
    - Size is calculated by subtracting the starting sector address of the data area from the total number of sectors in the file system (specified in the boot sector).
    - organized into clusters, and the number of sectors per cluster is given in the boot sector.

ESSENTIAL BOOT SECTOR DATA

- FAT12/16 root directory has a fixed size that is given in the boot sector
- Starting address for the FAT32 root directory is given in the boot sector, and size is determined from the FAT structure
- Dynamic size and location of the root directory allows FAT32 to
  - adapt to bad sectors in the beginning of the data area
  - grow as large as it needs to



NON-ESSENTIAL BOOT SECTOR DATA

- an 8-character string called the OEM name that represent the tool used to make the file system (optional)
- a 4-byte volume serial number that is, according to the Microsoft specification, determined at file system creation time by using the current time
- an 8-character type string "FAT12“, "FAT16“, "FAT32“, or "FAT" (can’t trust)
- an 11-character volume label string that the user specify while creating the file system.
- volume label is also saved in the root directory of the file system

BOOT CODE

- Boot sector is 512 bytes
- First 3 bytes of the boot sector contain a jump instruction in machine code that causes the CPU to jump past the configuration data to the rest of the boot code.
- Bytes 62 to 509 in FAT12/16 and bytes 90 to 509 in FAT32 contain the boot code and FAT32 can use the sectors following the boot sector for additional boot code.
- common for FAT file systems to have boot code even though they are not bootable and only displays a message to show that another disk is needed to boot the system.

ANALYSIS TECHNIQUES

- Purpose of analyzing the file system category of data: to determine the file system layout and configuration details.
- Example: which OS formatted the disk or hidden data.
- To determine the configuration of a FAT file system, we need to locate and process the boot sector
- Using the information from the boot sector, we can calculate the locations of the reserved area, the FAT area, and the data area.
- The FAT32 FSINFO data structure provide clues about recent activity, and its location is given in the boot sector
- Backup copies of both data structures also exist in FAT32.

ANALYSIS CONSIDERATIONS

- no values show when or where the file system was created, but the OEM label and volume label (nonessential), may give some clues because different tools have different default values.
- a special file that has a name equal to the volume label, and it might contain the file system creation time.
- Hidden data locations:
  - Over 450 bytes of data between the end of the boot sector data and the final signature, generally used to store boot code, but not needed for non-bootable file systems.
  - FAT32 file systems typically allocate many sectors to the reserved area, but only a few are used for the primary boot sector, the backup boot sector, and the FSINFO data structure.
  - FAT32 FSINFO data structure has hundreds of unused bytes.
  - Volume slack (space between the end of the file system and the end of the volume)

ANALYSIS CONSIDERATIONS

- OS generally wipes the sectors in the reserved area when it creates the file system.
- Relatively easy to create volume slack because we need to modify only the total number of sectors value in the boot sector.
- Compare the number of sectors in the file system, given in the boot sector, with the number of sectors that are in the volume to find volume slack.
- Compare the primary and backup boot sectors of FAT32 file systems to identify inconsistencies.

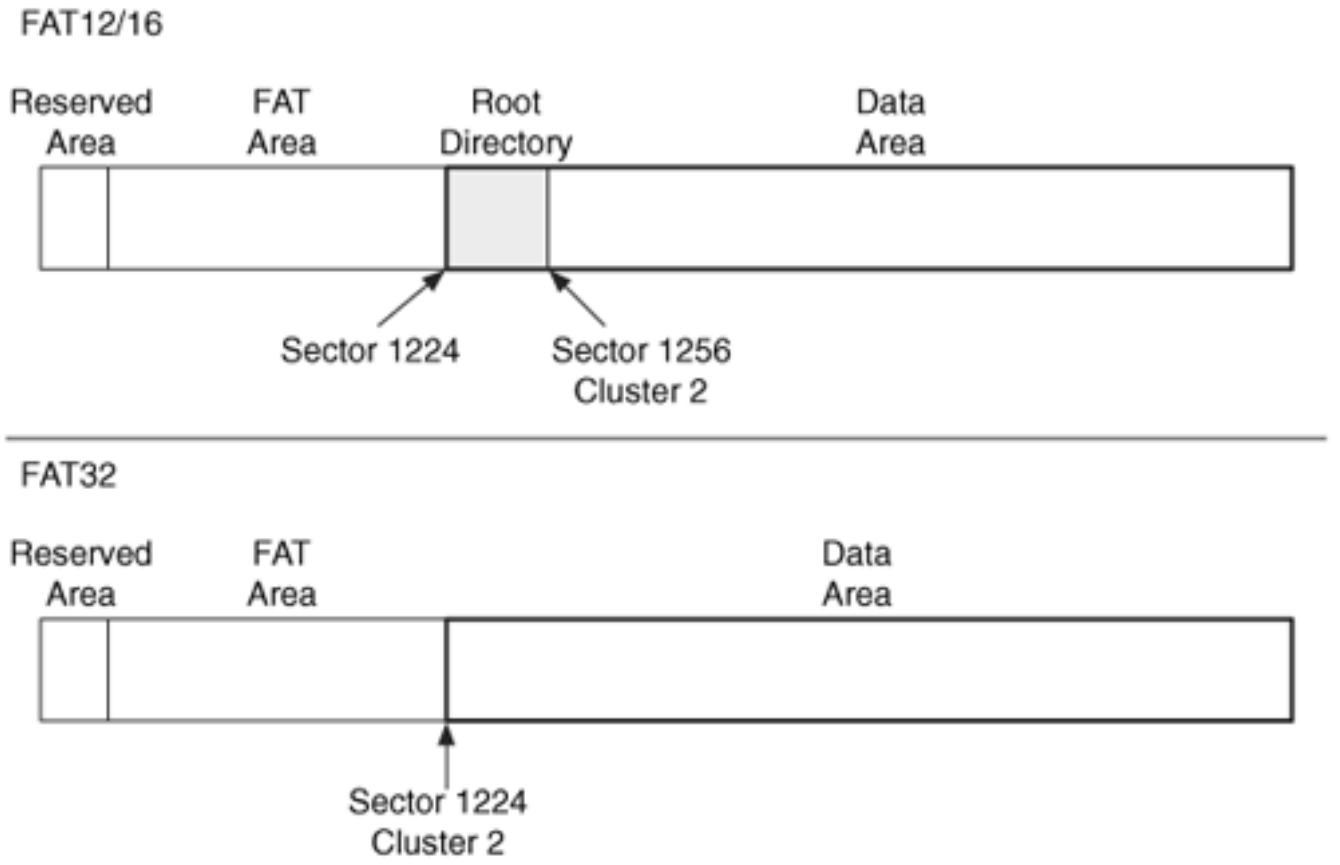
CONTENT CATEGORY

CONTENT CATEGORY

- includes the data that comprise file or directory content.
- uses the term cluster for its data units.
- Cluster: a group of consecutive sectors, and number of sectors must be a power of 2
- maximum cluster size is 32KB (Microsoft specification)
- address of the first cluster is 2. There are no clusters that have an address of 0 or 1.
- All clusters are located in the data area of the file system

FINDING THE FIRST CLUSTER

- Finding the location of the first cluster is harder because it is not at the beginning of the file system; it is in the data area.
- Reserved and FAT areas do not use cluster addresses.
- In FAT file system, not every logical volume address has a logical file system address.
- In NTFS, first cluster is also the first sector of the file system.



CLUSTER AND SECTOR ADDRESSES

- Cluster addresses do not start until the data area.
- To address the data in the reserved and FAT areas, we use the sector address (logical volume address).
- Calculating the sector address of cluster C:

$$S = (C - 2) * (\#\text{ of sectors per cluster}) + (\text{sector of cluster 2})$$

- Translate a sector S to a cluster:

$$C = ((S - \text{sector of cluster 2}) / (\#\text{ of sectors per cluster})) + 2$$

CLUSTER ALLOCATION STATUS

- Allocation status of a cluster is determined using the FAT structure.
- two copies of the FAT, and the first one starts after the reserved area
- FAT has one entry for every cluster in the file system
- Size of FAT entry is dependent on the version of FAT
  - 12 (FAT12), 16 (FAT16), 32, only 28 bits are used (FAT32)
- Each entry represents the allocation status of each cluster:

FAT12	FAT16	FAT32	Meaning
0x000	0x0000	0x00000000	Free Cluster
0xff7	0xfff7	0x0ffffff7	Bad Cluster
0xff8 or greater	0xfff8 or greater	0x0ffffff8 or greater	End of File
Any other hex value			Next Cluster

ALLOCATION ALGORITHMS

- OS gets to choose which allocation algorithm it uses when it allocates the clusters.
- Windows 98 and Windows XP: next available algorithm
- To find an unallocated cluster, the OS scans the FAT for an entry that has a 0 in it or
- If FAT32 file system, identify the next free cluster from the FSINFO data structure in the reserved area
- To change a cluster to unallocated status, the corresponding entries in the FAT structures are located and set to 0.
- Most Oses do not clear the cluster contents when it is unallocated unless they implement a secure wiping feature.

ANALYSIS TECHNIQUES

- To locate a specific data unit
  - to locate a data unit prior to the start of the data area, we need to use sector addresses.
  - For the data units in the data area, we can use either sector or cluster addresses.
- determine the allocation status
  - look at the cluster's entry in the FAT (0 unallocated and non-zero allocated)
  - to extract the contents of all unallocated clusters, read the FAT and extract each cluster with a 0.
  - Data units prior to the data area are not listed in the FAT and hence do not have an official allocation state, although most are used by the file system.
- do something with the content

ANALYSIS CONSIDERATIONS

1. Clusters marked as bad should be examined because many disks handle bad sectors at the hardware level, and the OS does not see them.
2. If the size of the data area is not a multiple of the cluster size, the sectors at the end of the data area that are not part of a cluster could be used to hide data or could contain data from a previous file system.
   - To determine if there are unused sectors, use the equation below and if there is a remainder from the division, there are unused sectors:
     (Total number of sectors – Sector address of cluster 2) / (Number of sectors per cluster)

ANALYSIS CONSIDERATIONS

3. Data could be hidden between the end of the last valid entry in the primary FAT structure and the start of the backup copy and between the end of the last entry in the backup FAT and the start of the data area.
   - To calculate how much unused space there is, compare the size of each FAT, given in the boot sector, with the size needed for the number of clusters in the file system.
   - Example: Suppose there are 797 sectors allocated to each FAT in the FAT32 file system and each table entry is four bytes and, therefore, 128 entries exist in each 512-byte sector. Each table has room for

     797 sectors * 128 (entries / sector) = 102,016 entries
   - **fsstat** output shows that there are 102,000 clusters, so there are 16 unused table entries for a total size of 64 bytes.

ANALYSIS CONSIDERATIONS

4. Not every sector in the volume is assigned a cluster address, so the results from doing a logical volume search versus a logical file system search may be different.
   - Test your tools to see if they search the boot sector and FAT areas by searching for the phrase "FAT" and see if it finds the boot sector (first verify that your boot sector has the string in it).

METADATA CATEGORY

METADATA CATEGORY

- includes the data that describe a file or directory, including the locations where the content is stored, dates and times, and permissions.
- to obtain additional information about a file or to identify suspect files.
- In a FAT file system, this information is stored in a directory entry structure.
- FAT structure also is used to store metadata information about the layout of a file or directory.
- In the FAT file system, a directory is considered a special type of file.

DIRECTORY ENTRIES

- a 32-byte data structure that is allocated for every file and directory.
- contains the file's attributes, size, starting cluster, and dates and times.
- can exist anywhere in the data area because they are stored in the clusters allocated to a directory.
- When a new file or directory is created, a directory entry in the parent directory is allocated for it.
- imagine the contents of a directory to be a table of directory entries.
- not given unique numerical addresses. Instead, use the full name of the file or directory that allocated it to address a directory entry.

DIRECTORY ENTRIES

- Attributes field of directory entry structure has 7 file attributes that can be set for each file
- Essential:
  - **directory** attribute: identify the directory entries that are for directories. The clusters allocated to a directory entry should contain more directory entries.
  - **long file name** attribute: identifies a special type of entry that has a different layout.
  - **volume label**: only one directory entry, by specification, should have this attribute set.
- If none of these attributes are set, the entry is for a normal file.

DIRECTORY ENTRIES

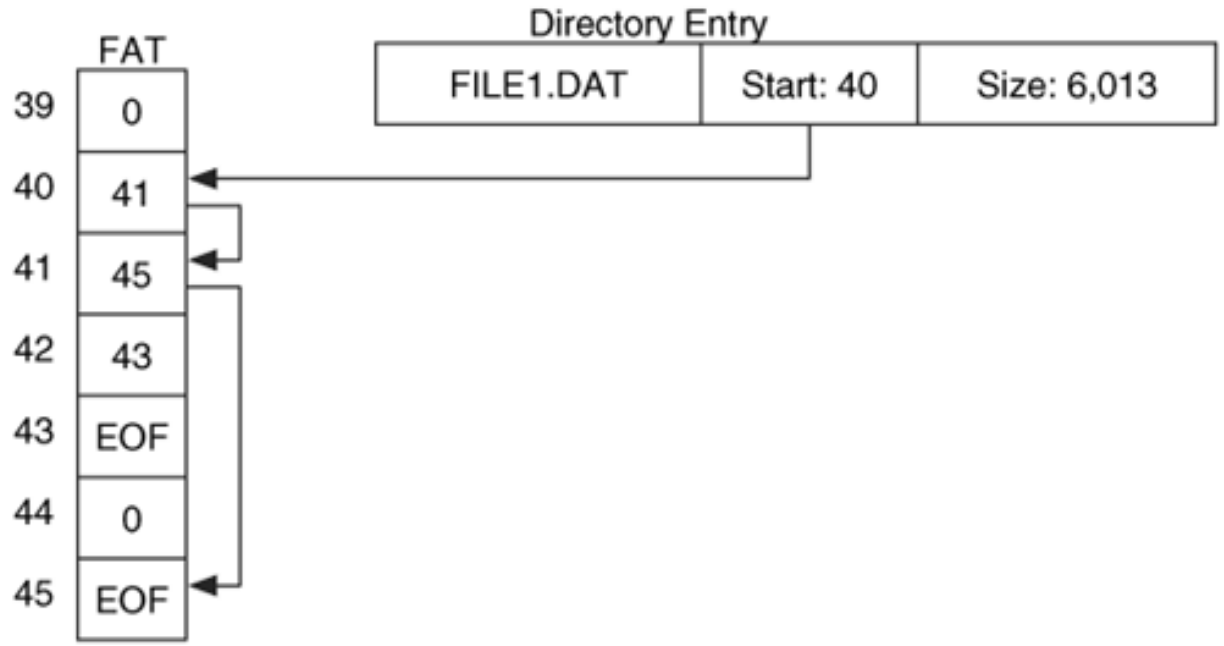
- Non-essential:
  - **read only** attribute should prevent a file from being written to, but the directories in Windows XP and 98 can have new files created in them when they are set to read only.
  - **hidden** attribute may cause files and directories to not be listed, but there is typically a setting in the OS that causes them to be shown.
  - **system** attribute should identify a file as a system file
  - **archive** attribute is set by Windows when a file is created or written to. The purpose of this attribute is that a backup utility can identify which files have changed since the last backup.

DIRECTORY ENTRIES

- Each directory entry has three times in it:
  - Created (optional): accurate to a tenth of a second
  - last accessed (optional): accurate to the day
  - last written: accurate to two seconds.
- There are no specifications for which operations cause each time to be updated, so each OS that uses FAT has its own policy with respect to when it updates the times.
- Times are stored with respect to the local time zone
- Allocation status of a directory entry is determined by using the first byte.
  - With an allocated entry, the first byte stores the first character in the file name
  - For an unallocated entry, the first byte will be 0xe5

CLUSTER CHAINS

- Directory entry contains the starting cluster of the file, and the FAT structure is used to find the remaining clusters in the file



CLUSTER CHAINS

- To find the next cluster in a file, you simply look at the cluster's entry in the FAT
  - If it is non-zero, it is allocated and contains the address of the next cluster in the file
  - an end of file marker if it is the last cluster in the file
  - a value to show that the cluster has bad sectors.
- Sequence of clusters is sometimes called a cluster chain
- Each entry has a maximum cluster address that it can store for the next cluster in the chain.
- Maximum size of a FAT file system is based on the size of the FAT entries.

DIRECTORIES

- When a new directory is created, a cluster is allocated to it and wiped with 0s.
- Size field in the directory entry is always 0. To determine the size of the directory, use the starting cluster from the directory entry and follow the cluster chain in the FAT structure until the end of file marker is found.
- First two directory entries in a directory are for the . (current) and .. (parent) directories with directory attribute set
- Creation date on a directory should be the same value as the . and .. entries

Cluster 110

Name	Created	Cluster
dir2	3/30/04 01:29:01	128
dir1	4/03/04 11:47:40	196
file8.dat	3/30/04 20:41:12	112

Cluster 196

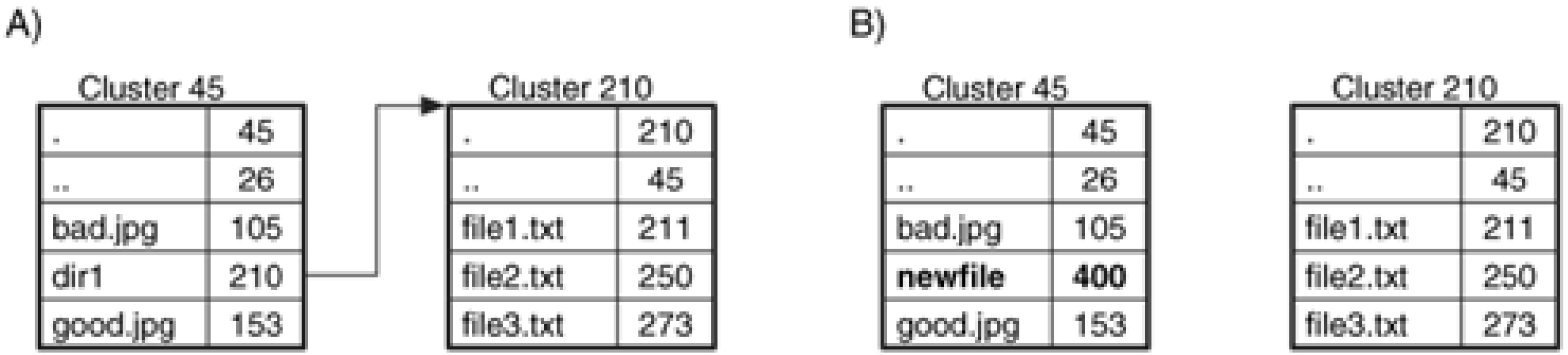
Name	Created	Cluster
.	4/1/04 09:27:00	196
..	4/1/04 09:27:00	110
file1.dat	4/3/04 12:58:23	297

DIRECTORY ENTRY ADDRESSES

- only standard way to address a directory entry is to use the full name of the file or directory that allocated it.
- Normal Scenario: find all allocated directory entry structures
  - start in the root directory and recursively look in each of the allocated directories.
  - For each directory, we examine each 32-byte structure and skip over the unallocated ones.
  - address of each structure is the name of the directory that we are currently looking at plus the name of the file
- Problem 1: unique naming conflict while finding unallocated directory entries
  - first letter of the name is deleted when the directory entry is unallocated.
  - If a directory had two files A-1.DAT and B-1.DAT that were deleted, both entries would have the same name in them, _-1.DAT.

DIRECTORY ENTRY ADDRESSES

- Problem 2: when a directory is deleted and its entry is reallocated
  - No pointer to the files and directories in the deleted directory
  - Orphan files: unallocated directory entries in the directory's allocated cluster still exist, but we cannot find them by simply following the directory tree, and if we do find them, we do not have an address for them.

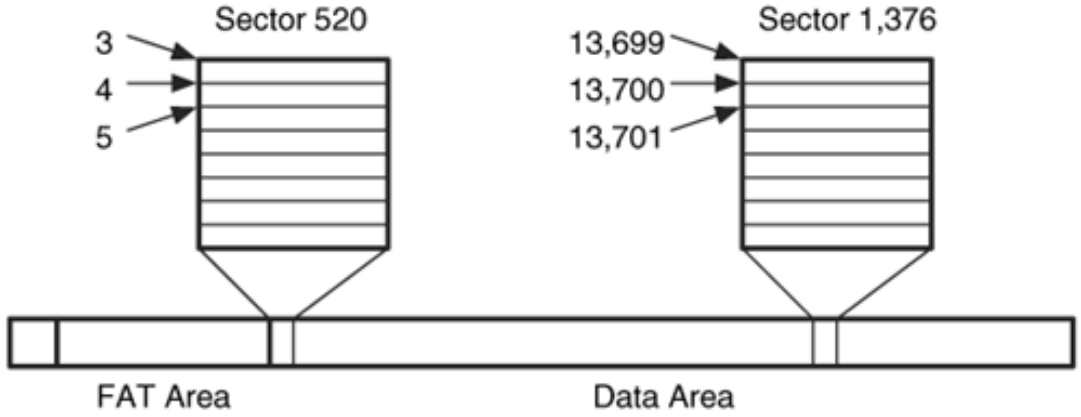


FINDING ORPHAN FILES

- Technique 1:
  - examine the first 32-bytes of each sector, not cluster in the data area
  - compare it to the fields in a directory entry.
  - If they are in a valid range, the rest of the sector should be processed.
  - may find entries in the slack space of clusters that have been allocated to other files.
- Technique 2:
  - search the first 32 bytes of each cluster to find the . and .. entries that are the first two entries in every directory.
  - will only find the first cluster of a directory and not any of its fragments.

SOLVING THE 2 PROBLEMS: NEW ADDRESSING

- TSK uses a method that is also used by several Unix systems
  - assume that every cluster and sector could be allocated by a directory.
  - every sector is divided into 32-byte entries that could store a directory entry, and assign a unique address to each entry.
  - Problem: Every directory and file will have an address except for the root directory.
  - The fix that TSK uses is to assign the root directory with an address of 2 and assign the first entry in the first sector an address of 3.
  - Each 512-byte sector can store 16 directory entry structures

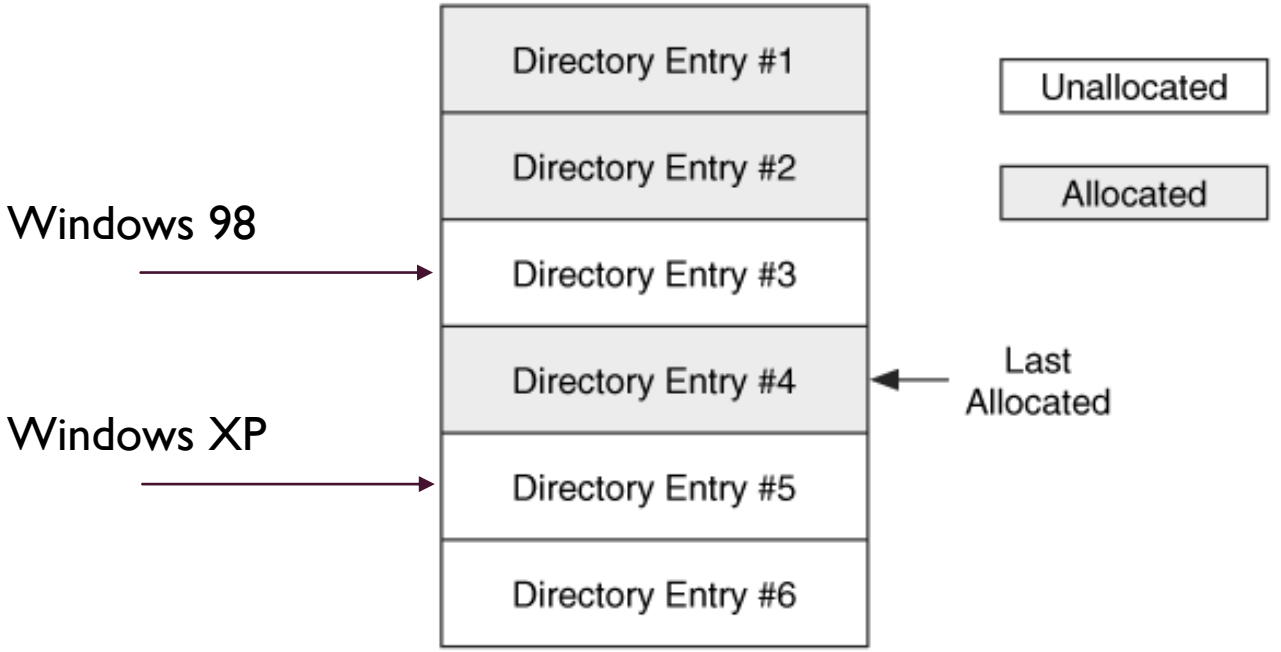


ALLOCATION ALGORITHMS

- In the metadata category, there are two general types of data that are allocated.
  - directory entries need to be allocated for new files and directories. First, search for an unallocated directory entry in a directory. If not found, a new cluster is allocated
  - temporal information for each file and directory needs to be updated.
- OS-dependent

DIRECTORY ENTRY ALLOCATION

- Windows 98 uses a first-available allocation strategy.
- Windows XP uses a next-available allocation method and restarts its scan from the beginning of the directory when it gets to the end of the cluster chain.



DIRECTORY ENTRY ALLOCATION

- When a file is renamed in Windows, a new entry is made for the new name, and the entry for the old name is unallocated.
- When a file is created from a Windows XP application, two entries are created.
  - The first entry has all the values except the size and the starting cluster.
  - This entry becomes unallocated, and a second entry is created with the size and starting cluster.
  - This occurs when saving from within an application, but not from the command line, drag and dropping, or using the 'New' menu option.
- When a file is deleted,
  - first byte of the directory entry is set to 0xe5.
  - clusters that were allocated for the file are unallocated by setting their corresponding entries in the FAT structure to 0.
  - Windows keeps the starting cluster of the cluster chain in the directory entry so some file recovery can be performed until the clusters are allocated to new files and overwritten.

TIME VALUE UPDATING

- 3 time values in a directory entry: last accessed, last written, and created.
- **Creation time** corresponds to the time when the first directory entry was allocated for the file, regardless of the original location
  - set when Windows allocates a new directory entry for a new file.
  - if the OS allocates a new directory entry for an existing file, even if original location was on a different disk, the original creation time is kept.
  - Example: if a file is renamed or is moved to another directory or disk, the creation time from the original entry is written to the new entry.
  - If a file is copied, a new file is being created, and a new creation time is written to the new entry.

TIME VALUE UPDATING

- **Written time** is content-based and not directory entry-based and follows data as it is copied around
  - set when Windows writes new file content.
  - If files are moved or copied in Windows, the new directory entry has the written time from the original file.
  - Changing the attributes or name of a file does not result in an update to this time.
  - Windows updates this time when an application writes content to the file, even if the application is doing an automatic save and no content has changed.

TIME VALUE UPDATING

- Last accessed date, which is only accurate to the day, is updated more frequently.
  - Anytime the file is opened, the date is updated.
  - When you right-click on the file to see its properties, its access date is updated.
  - If you copy or move a file, the access date on both the source and destination is updated.
  - If you move a file to a new volume, the access date on the new file is updated because Windows had to read the original content before it could write it to the new volume.
  - If you move a file within the same volume, though, the access date will not change because the new file is using the same clusters.

TIME VALUE UPDATING

- If you move a file in Windows, resulting file will have the original written and original creation time unless the move is to a different volume and the command line is used.
- If you copy a file, resulting file will have the original written time and a new creation time. **confusing because the creation date is after the written time**.
- If you create a new file from an application in Windows, it is common for the written time to be a little later than the creation time.
- For directories, dates were set when the directory was created and won't be updated after that. Even when new clusters were allocated for the directory or new files were created in the directory, the written times were not updated.

ANALYSIS TECHNIQUES

- metadata analysis: to determine more details about a file or directory.
- locate a specific directory entry, use the starting cluster from the directory entry, and use FAT structure to identify all clusters that the file or directory has allocated.
- Locating all entries in FAT is difficult because only allocated directory entries have a full address, which is their path and file name.
- Unallocated entries might not have full addresses; therefore, a tool-specific addressing scheme will likely be used.

ANALYSIS CONSIDERATIONS

- Directory entry time values
  - times are stored without respect to time zone, so it does not matter what time zone your analysis system is set to.
  - dealing with daylight savings easier
  - last accessed and the created dates and times are optional by specification and might be 0
  - Copying a file can result in a creation time that corresponds to the time that the copy occurred and a written date that corresponds to the last written date of the original location. This type of scenario is fairly common and not necessarily a sign of tampering.

ANALYSIS CONSIDERATIONS

- Allocation routines
  - allocation routines for new directory entries in an XP system allow you to see the names of deleted files for longer because a first available algorithm is not used. However, you might not recover the original file content even if you can see the name because the clusters of the file will be reallocated just as quickly.
  - DEFRAG utility in Windows compacts directories and removes unused directory entries. It also moves the file content around to make it more contiguous. Therefore, recovery after running DEFRAG is difficult.
  - Content of slack space is OS-dependent. Unused sectors in the last cluster of a file allocated by Windows typically contain deleted data. It is rare that FAT specification clear all sectors when it allocates a cluster to a file

ANALYSIS CONSIDERATIONS

- Consistency checks
  - Windows does not show any data after it finds a directory entry with all zeros. Someone can create a directory with only a few files and use the rest of the directory space for hiding data. Compare the allocated size of a directory to the number of allocated files. A logical file system search should find any data that is hidden in these locations.
  - Access and created times in the volume label directory entries are frequently set to 0, but the last written time is usually set by Windows to the current time when the file system is created.
  - With Windows XP, a volume label directory entry can be used to hide data by manually editing the starting cluster field to contain the address of an unused cluster and FAT to allocate a cluster chain. ScanDisk program will not raise any warnings or errors. In Windows 98, ScanDisk will detect that clusters have been allocated to the volume label and remove them.
  - In Windows XP, you can have as many volume label directory entries as you want and in any location for hiding data.

FILE NAME CATEGORY

FILE NAME CATEGORY

- Analysis in the file name category map a file name with a metadata structure.
- FAT does not separate the file name address and metadata address
- Directory entry data structure contains the name of a file using the 8.3 naming convention, where the file name has 8 characters and the extension has 3 characters.
- Long File Name (LFN)
  - If a file is given a name that is longer than eight characters or if the name has special values, LFN type of directory entry is added.
  - Files with an LFN also have a normal, short file name (SFN), directory entry.
  - LFN entries only contain the file name and not the time, size, or starting cluster information.
  - When an LFN entry becomes unallocated, the first byte in the data structure is set to 0xe5.

LONG FILE NAME (LFN)

- SFN and LFN data structures have an attribute field in the same location, and LFN entries use a special attribute value.
- Remaining bytes in the entry are used to store 13 Unicode characters encoded in UTF-16, which are 2 bytes each.
- If a file name needs more than 13 characters, additional LFN entries are used.
- All LFN entries precede the SFN entry in the directory and contain a checksum that can be used to correlate the LFN entries with the SFN entry.
- LFN entries are in reverse order so that the first part of the file name is closest to the SFN entry.

```
Atr: File  Name: RESUME-1.RTF Cluster: 9
Atr: LFN   Seq: 2  CSum: 0xdf  Name: Name.rtf
Atr: LFN   Seq: 1  CSum: 0xdf  Name: My Long File
Atr: File  Name: MYLONG~1.RTF Cluster: 26
Atr: File  Name: _ILE6.TXT Cluster: 48
```

```
# fls -f fat fat-2.dd
r/r 3: FAT DISK (Volume Label Entry)
r/r 4: RESUME-1.RTF
r/r 7: My Long File Name.rtf (MYLONG~1.RTF)
r/r * 8: _ile6.txt
```

ALLOCATION ALGORITHMS

- file names are stored in the same data structures as the metadata, so the allocation algorithms are also same.
- Difference: LFN entries must come before the SFN, so the OS will look for enough room for all entries.
- Same deletion routine:
  - first byte of the entry is replaced with 0xe5.
  - With an SFN entry, this will overwrite the first character of the name and an LFN directory entry will have its sequence number overwritten.
  - When we recover deleted files, we need to supply the first character of the short name, but if the file also has a long name, we can use its first character.

ANALYSIS TECHNIQUES

- To find a specific file name.
  - first locate the root directory
  - process the contents of a directory by stepping through each 32-byte directory entry structure.
  - If the attribute for the structure is set for an LFN entry, its contents are stored and the next entry is examined.
  - This process repeats until we find an entry that is not an LFN and whose checksum matches that of the long name entries.
  - allocation status of an entry is determined using the first byte
- When we find a file or directory of interest, we look for the corresponding metadata.
- Easy with FAT because all the metadata for a file is located in the directory entry.
- Metadata associated with a file name will not become out of sync after file deletion.
- Metadata associated with an unallocated file name will be accurate until both are overwritten.

ANALYSIS CONSIDERATIONS

- Because the file name and metadata are in the same data structure, we will always have a name for unallocated entries.
- If LFNs are used, the entire long name can be recovered, although we may lose part of the name when the first byte is changed to 0xe5.
- In Windows XP, we can create LFN entries that did not correspond to any SFN entries, and no error was given. This can hide small amounts of data that are not shown when the directory contents were listed.

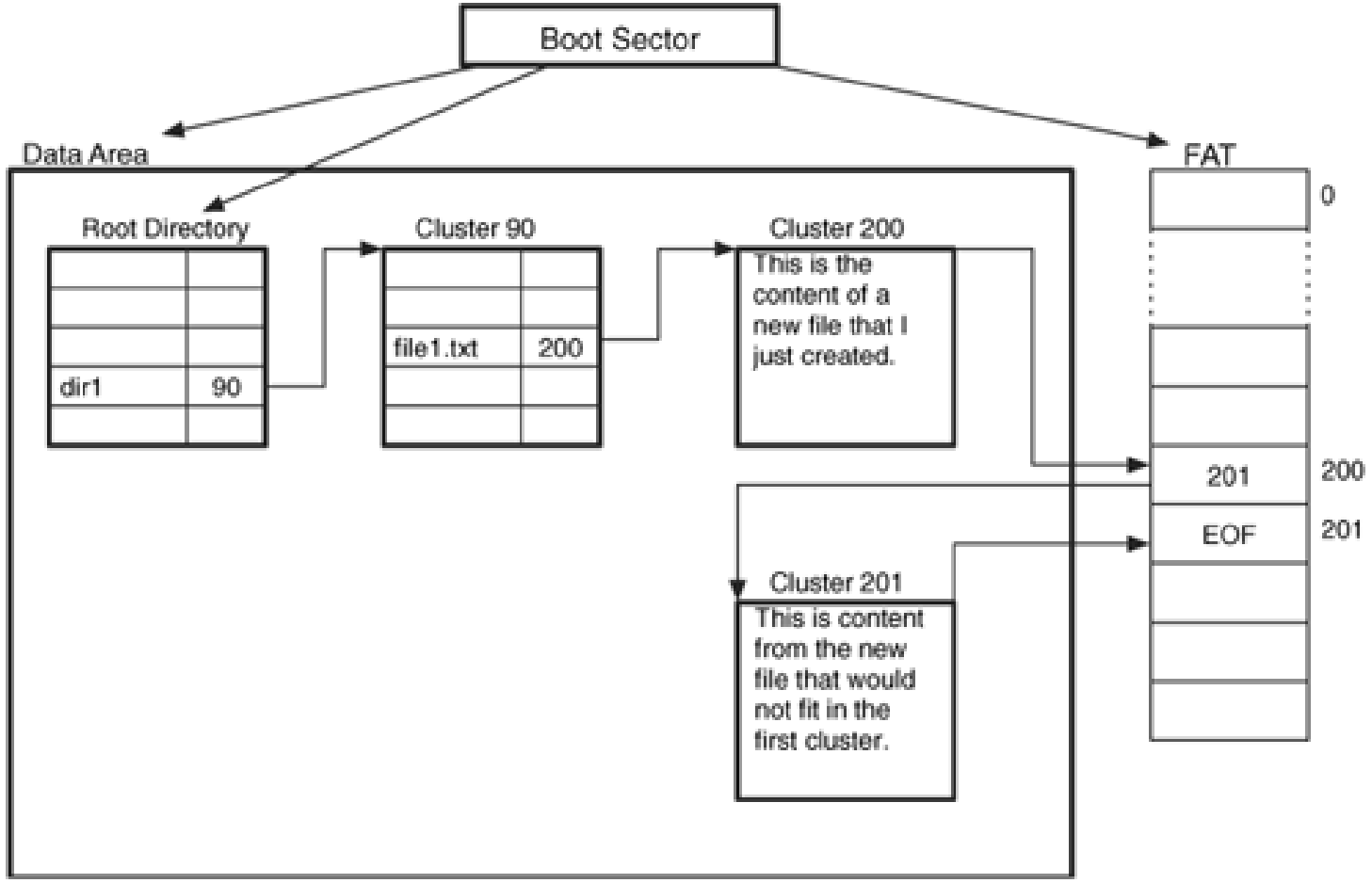
ANALYSIS CONSIDERATIONS

- can hide data at the end of a directory.
  - Some OSes will stop processing directory entries after finding one that is all zeros.
  - Data could be hidden after that all zero entry
- If you are using the keyword search mechanism of a search tool to look for a file name
  - use the Unicode version of long file names.
  - SFN is stored in ASCII, but the LFN entries are in Unicode.
  - entire name might not be stored sequentially, and it might be broken up into smaller pieces in multiple parts of multiple LFN entries

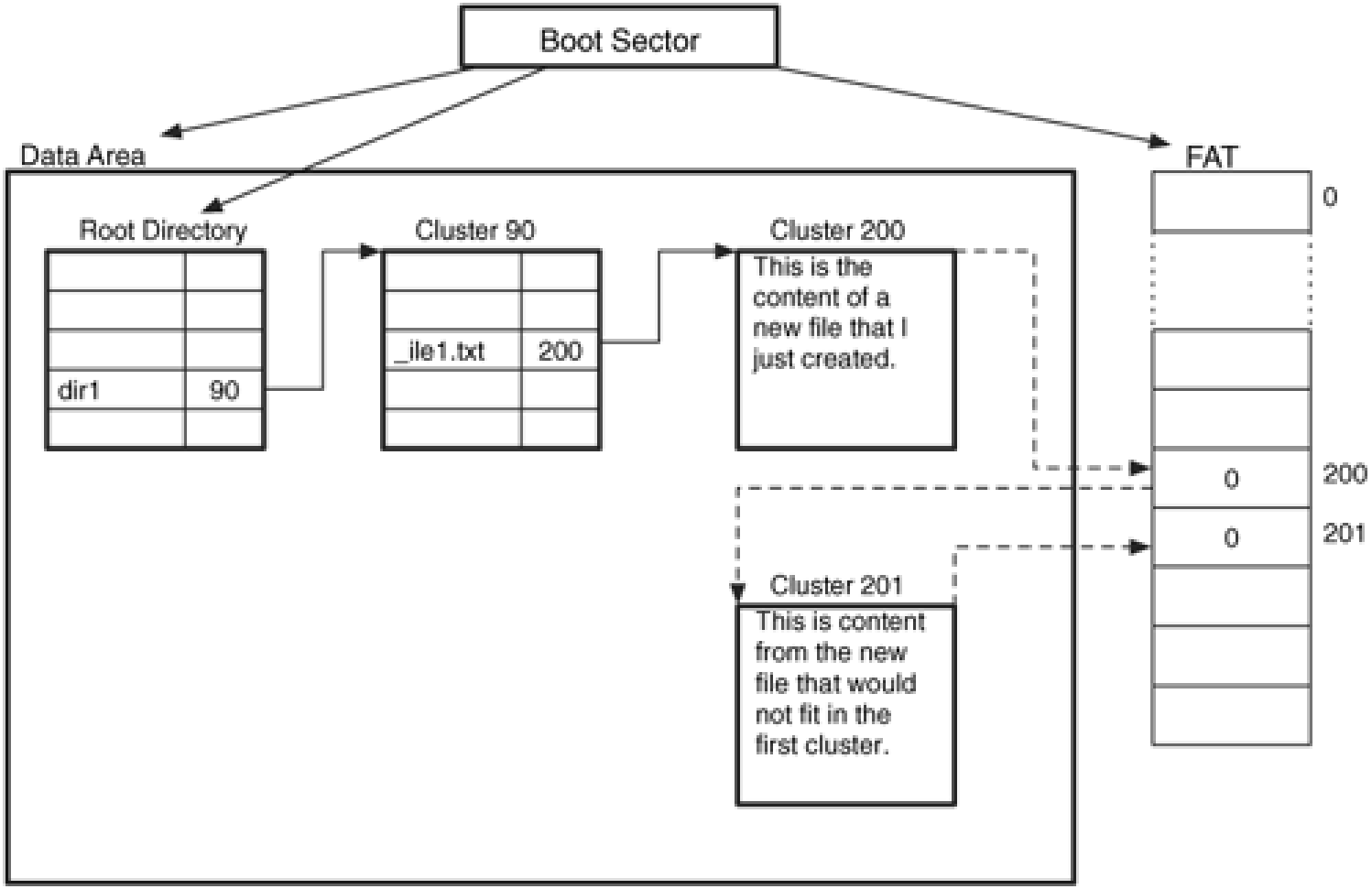
FILE ALLOCATION EXAMPLE

Creation of dir1\file1.dat file.

- dir1 directory already exists
- cluster size: 4,096 bytes
- file size: 6,000 bytes.

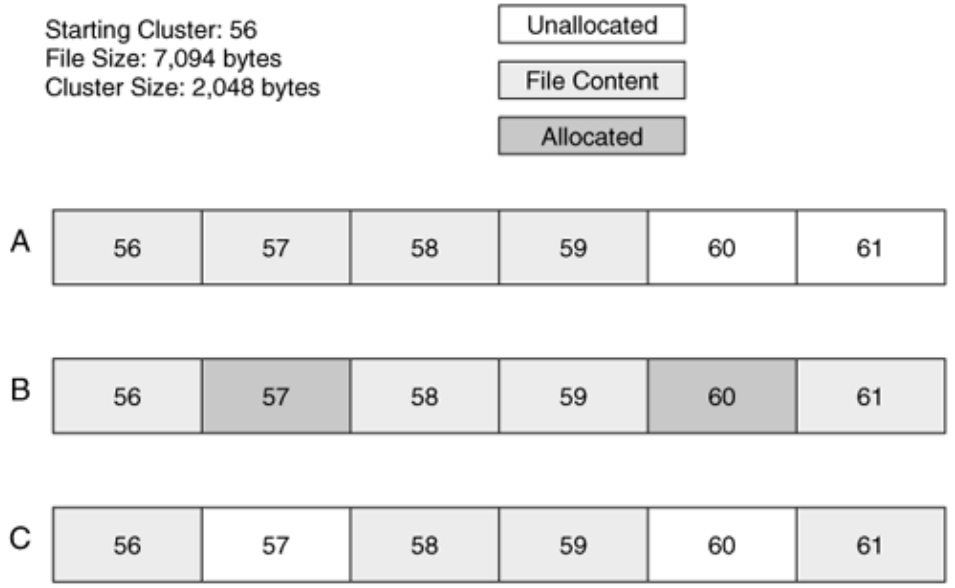


FILE DELETION EXAMPLE



FILE RECOVERY

- To recover a file, we have the starting location and the size of the file.
- Start reading the data from the known starting cluster.
- A recovery tool (or person) has two options for choosing the remaining clusters:
  - blindly read the amount of data needed for the file size, ignoring the allocation status
  - read only from the unallocated clusters. More successful in recovering fragmented files



FILE RECOVERY

- Scenario A
  - 4 clusters of the file are consecutive.
  - Both options will correctly recover clusters
- Scenario B
  - file was fragmented into three chunks and the clusters in between the fragments, clusters 57 and 60, are still allocated to another file when the recovery takes place.
  - Option 1 will recover clusters 56 to 59 and incorrectly include the contents of cluster 57.
  - Option 2 will correctly recover clusters 56, 58, 59, and 61.
- Scenario C
  - file was fragmented into three chunks and the clusters in between the fragments, clusters 57 and 60, are not allocated when the recovery takes place.
  - Both options will incorrectly recover clusters 56 to 59

Option 2 which considers the allocation status of clusters can recover deleted files from more scenarios than option 1.

FILE RECOVERY

- Recovery of directories:
  - multiple cluster directories are likely to be fragmented.
  - second cluster is only allocated when it is needed and it is highly unlikely that the next consecutive cluster will be available (because that cluster was probably allocated to one of the files in the directory).
  - directories are harder to recover, unless the directory is the result of a copy or the file system has been recently defragmented.
  - When a directory is copied and its size is known, then consecutive clusters can be allocated for it.
- If defragmenting software is frequently run on the file system,
  - recovery of the files deleted after the last defragmenting will be easier because more of the files will be in consecutive clusters.
  - Files that were deleted prior to the defragmenting will be very difficult to recover because the clusters could have been reallocated.

DETERMINING THE FAT FILE SYSTEM TYPE

- determines the type based on the number of clusters in the file system.
- To get the number of clusters, we need to determine how many sectors there are in the data area and then divide that number by the number of sectors per cluster.
- For FAT12/16, root directory takes the first sectors of the data area, and cluster 2 starts after it.
- For FAT32, root directory is dynamic, and cluster 2 starts at the beginning of the data area.
- There is a value in the boot sector that identifies how many entries there are in the root directory, and for FAT32 this value is 0.

DETERMINING THE FAT FILE SYSTEM TYPE

- We can calculate the number of sectors in the root directory by multiplying the number of entries by 32 bytes (the size of each directory entry), dividing by the number of bytes per sector, and rounding up (if needed):

  ((NUM_ENTRIES * 32) + (BYTES_PER_SECTOR – 1)) / (BYTES_PER_SECTOR)

- Previous calculation will be zero for FAT32.
- We determine the number of sectors that are allocated to clusters by taking the total number of sectors in the file system and subtracting the size of the reserved area, the size of the FAT area, and the size of the root directory.

DETERMINING THE FAT FILE SYSTEM TYPE

- In other words, we are subtracting the sector address of cluster 2 from the total number of sectors.

  TOTAL_SECTORS – RESERVED_SIZE – NUM_FAT * FAT_SIZE – ROOTDIR_SIZE

- Divide the number of sectors in the data area by the number of sectors in a cluster.
- If this value is
  - < 4,085: FAT12
  - >= 4,085 and < 65,525: FAT16
  - >= 65,525: FAT32.
- These values correspond to the maximum number of clusters that a FAT can store.
- 8 reserved values for EOF, 1 for bad clusters, and the unused entries in slots 0 and 1.

CONSISTENCY CHECK

- Reserved area:
  - For the boot sector and other data structures in the reserved area of a FAT file system, a consistency check should
    - verify that the defined values are in the appropriate range
    - examine the unused locations for non-zero values. For example, there are many sectors in the reserved area that are not used in every file system.
  - Compare the primary and backup boot sectors in a FAT32 file system
- FAT area:
  - Compare the backup and primary FATs to verify that they have the same values.
  - Examine each entry that is marked as bad because most hard disks fix errors before the OS notices them.
  - Examine the space between the FAT entry for the last cluster and the end of the sector allocated to the FAT for each FAT because this space is not used by the file system and could contain hidden data.

CONSISTENCY CHECK

- Data area:
  - Space between the end of the last cluster and the end of the file system might exist that does not have a cluster address.
  - Examine the root directory and its subdirectories and check each cluster chain in the FAT to make sure that an allocated directory entry points to the start of it.
  - Do the reverse check to make sure that allocated directory entries point to allocated clusters.
  - If multiple directory entries point to a cluster chain, Microsoft recommends that both files be copied to a new location and the original versions deleted
  - Length of the cluster chain should be the number of clusters needed for the size of the file.

CONSISTENCY CHECK

- Directory Entry:
  - Any directory entries that are marked as volume labels should not have a starting cluster, and there should only be one volume label entry in the file system.
  - Examine the checksums for the allocated LFN directory entries and compare to the allocated SFN entry. If a corresponding SFN entry cannot be found, the LFN entries should be examined. This could be a result of a file system crash, using an OS that doesn't support long file names, or they could contain hidden data.
  - Any directory entries in a directory that are all zeros or random data and have allocated entries before and after it could be the result of a wiping tool. Also check if there are any directory entries after a null entry. Some OSes will not show entries after a null entry.

FAT DATA STRUCTURES

FIRST 36 BYTES OF THE FAT BOOT SECTOR

Byte Range	Description	Essential?
0–2	Assembly instruction to jump to boot code.	No (unless it is a bootable file system)
3–10	OEM Name in ASCII	No
11–12	Bytes per sector. Allowed values include 512, 1024, 2048, and 4096.	Yes
13–13	Sectors per cluster. Allowed values are powers of 2, but the cluster size must be <= 32KB	Yes
14–15	Size in sectors of the reserved area.	Yes
16–16	Number of FATs. Typically 2 for redundancy, but it can be one for some small storage devices.	Yes

FIRST 36 BYTES OF THE FAT BOOT SECTOR

Byte Range	Description	Essential?
17–18	Maximum number of files in the root directory for FAT12 and FAT16. This is 0 for FAT32 and typically 512 for FAT16.	Yes
19–20	16-bit value of number of sectors in file system. If the number of sectors is larger than can be represented in this 2- byte value, a 4-byte value exists later in the data structure and this should be 0.	Yes
21–21	Media type. 0xf8 should be used for fixed disks and 0xf0 for removable.	No
22–23	16-bit size in sectors of each FAT for FAT12 and FAT16. For FAT32, this field is 0.	Yes
24–25	Sectors per track of storage device.	No

FIRST 36 BYTES OF THE FAT BOOT SECTOR

Byte Range	Description	Essential?
26–27	Number of heads in storage device.	No
28–31	Number of sectors before the start of partition.	No
32–35	32-bit value of number of sectors in file system. Either this value or the 16-bit value above must be 0.	Yes

BOOT SECTOR REMAINDER FOR FAT12/16

Byte Range	Description	Essential?
0–35	See Boot Sector's first 36 bytes	Yes
36–36	BIOS INT13h drive number	No
37–37	Not used	No
38–38	Extended boot signature to identify if the next three values are valid. The signature is 0x29.	No
39–42	Volume serial number, which some versions of Windows will calculate based on the creation date and time.	No

BOOT SECTOR REMAINDER FOR FAT12/16

Byte Range	Description	Essential?
43–53	Volume label in ASCII. The user chooses this value when creating the file system.	No
54–61	File system type label in ASCII. Standard values include "FAT," "FAT12," and "FAT16," but nothing is required.	No
62–509	Not used.	No
510–511	Signature value (0xAA55).	No

BOOT SECTOR REMAINDER FOR FAT32

Byte Range	Description	Essential?
0–35	See Boot Sector's first 36 bytes	Yes
36–39	32-bit size in sectors of one FAT.	Yes
40–41	Defines how multiple FAT structures are written to. If bit 7 is 1, only one of the FAT structures is active and its index is described in bits 0–3. Otherwise, all FAT structures are mirrors of each other.	Yes
42–43	The major and minor version number.	Yes
44–47	Cluster where root directory can be found.	Yes

BOOT SECTOR REMAINDER FOR FAT32

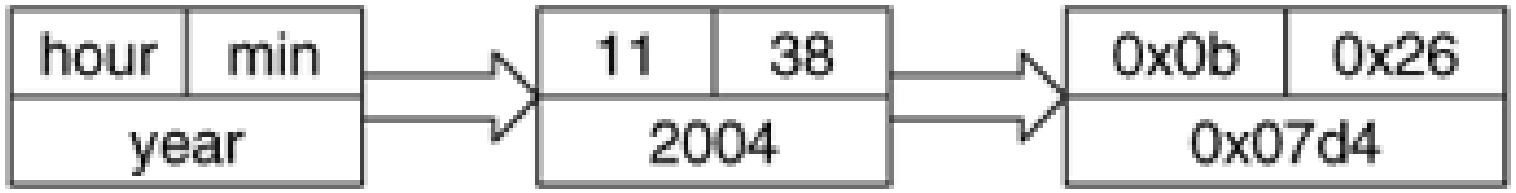
Byte Range	Description	Essential?
48–49	Sector where FSINFO structure can be found.	No
50–51	Sector where backup copy of boot sector is located (default is 6).	No
52–63	Reserved.	No
64–64	BIOS INT13h drive number.	No
65–65	Not used.	No
66–66	Extended boot signature to identify if the next three values are valid. The signature is 0x29.	No

BOOT SECTOR REMAINDER FOR FAT32

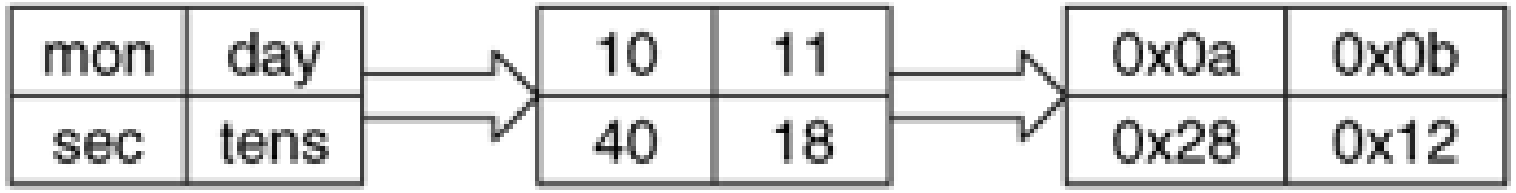
Byte Range	Description	Essential?
67–70	Volume serial number, which some versions of Windows will calculate based on the creation date and time.	No
71–81	Volume label in ASCII. The user chooses this value when creating the file system.	No
82–89	File system type label in ASCII. Standard values include "FAT32," but nothing is required.	No
90–509	Not used.	No
510– 511	Signature value (0xAA55).	No

VOLUME SERIAL NUMBER CALCULATION

October 11, 2004 11:38:40:18



0x0b26
+ 0x07d4
0x12fa



0x0a0b
+ 0x2812
0x321d

Serial Number: 12FA-321D

FAT32 FSINFO DATA STRUCTURE

- includes hints about where the OS can allocate new clusters.
- its location is given in the boot sector

Byte Range	Description
0–3	Signature (0x41615252)
4–483	Not used
484–487	Signature (0x61417272)
488–491	Number of free clusters
492–495	Next free cluster
496–507	Not used
508–511	Signature (0xAA550000)

FAT

- There are typically two FATs in a FAT file system, but the exact number is given in the boot sector.
- First FAT starts after the reserved sectors, the size of which is given in the boot sector.
- Total size of each FAT is also given in the boot sector
- Second FAT, if it exists, starts in the sector following the end of the first.
- Table consists of equal-sized entries and has no header or footer values.
- Size of each entry depends on the file system version.
  - FAT12 has 12-bit entries, FAT16 has 16-bit entries, and FAT32 has 32-bit entries.

FAT

- Entries are addressed starting with 0, and each entry corresponds to the cluster with the same address.
- First addressable cluster in the file system is #2.
- Entries 0 and 1 in the FAT structure are not needed.
  - Entry 0 typically stores a copy of the media type in boot sector
  - Entry 1 typically stores the dirty status of the file system

DIRECTORY ENTRIES

Byte Range	Description	Essential?
0–0	First character of file name in ASCII and allocation status (0xe5 or 0x00 if unallocated)	Yes
1–10	Characters 2 to 11 of file name in ASCII	Yes
11–11	File Attributes	Yes
12–12	Reserved	No
13–13	Created time (tenths of second)	No
14–15	Created time (hours, minutes, seconds)	No
16–17	Created day	No
18–19	Accessed day	No

DIRECTORY ENTRIES

Byte Range	Description	Essential?
20–21	High 2 bytes of first cluster address (0 for FAT12 and FAT16)	Yes
22–23	Written time (hours, minutes, seconds)	No
24–25	Written day	No
26–27	Low 2 bytes of first cluster address	Yes
28–31	Size of file (0 for directories)	Yes

- If the name does not have 8 characters, unused bytes are typically filled in with the ASCII value for a space, which is 0x20.
- File size field is 4 bytes and, therefore, the maximum file size is 4GB.
- Directories will have a size of 0 and the FAT structure is used to find the number of clusters allocated to it.

FLAG VALUES FOR THE DIRECTORY ENTRY ATTRIBUTES FIELD

Flag Value (in bits)	Description	Essential?
0000 0001 (0x01)	Read only	No
0000 0010 (0x02)	Hidden file	No
0000 0100 (0x04)	System file	No
0000 1000 (0x08)	Volume label	Yes
0000 1111 (0x0f)	Long file name	Yes
0001 0000 (0x10)	Directory	Yes
0010 0000 (0x20)	Archive	No

- Upper two bits of the attribute byte are reserved

LFN FAT DIRECTORY ENTRY

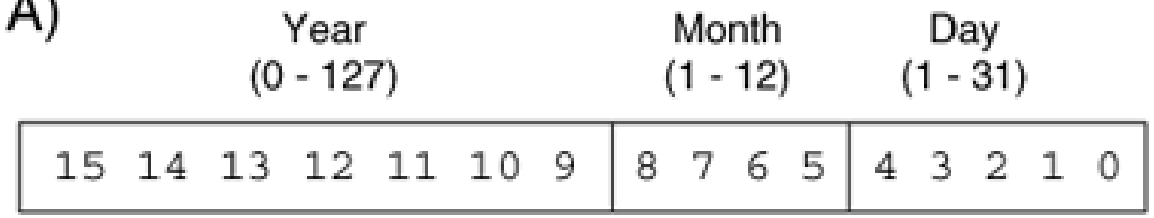
Byte Range	Description	Essential?
0–0	Sequence number (ORed with 0x40) and allocation status (0xe5 if unallocated)	Yes
1–10	File name characters 1–5 (Unicode)	Yes
11–11	File attributes (0x0f)	Yes
12–12	Reserved	No
13–13	Checksum	Yes
14–25	File name characters 6–11 (Unicode)	Yes
26–27	Reserved	No
28–31	File name characters 12–13 (Unicode)	Yes

- Checksum is calculated using the short name by iterating over each letter, and at each step rotating the current checksum by one bit to the right and then adding the ASCII value of the next letter.

BREAKDOWN OF THE DATE PORTION OF EACH TIMESTAMP

- 16-bit value that has three parts
- Year value in bits 9 to 15 is added to 1980
- Valid range is from 0 to 127, which gives a year range of 1980 to 2107

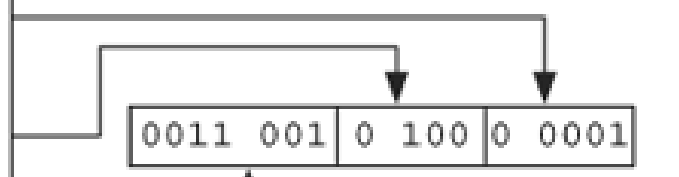
A)



B)

Start: April 1, 2005

Normal	Decimal	Hex	Binary
1	1	0x01	0 0001
April	4	0x04	0100
2005	25	0x19	001 1001

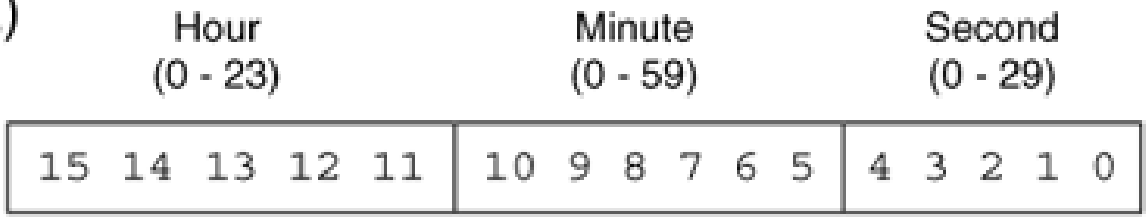


End: 0x3281

BREAKDOWN OF THE TIME PORTION OF EACH TIMESTAMP

- uses two-second intervals.
- Valid range is 0 to 29, which allows a second range of 0 to 58

A)



B)

Start: 10:31:44 AM

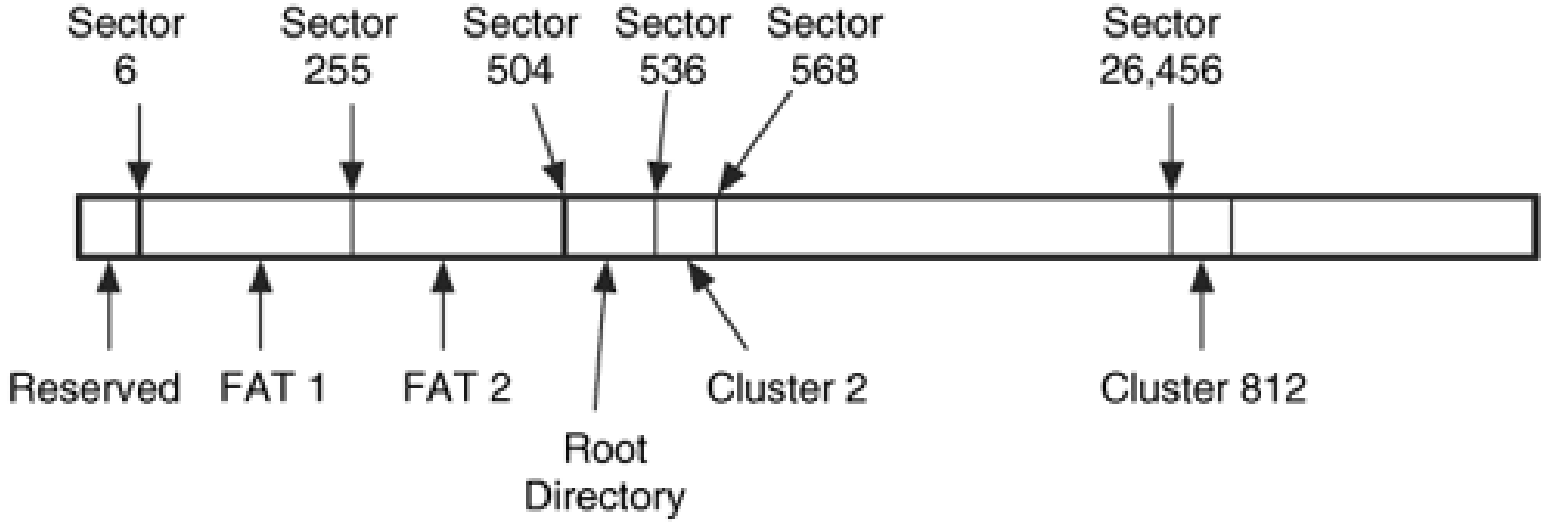
Normal	Decimal	Hex	Binary
44	22	0x16	1 0110
31	31	0x01f	01 1111
10	10	0x0a	0 1010

End: 0x53f6

ANALYSIS SCENARIOS

ANALYSIS SCENARIO: CONTENT CATEGORY

Locate the first sector of cluster 812 in a FAT16 file system. All we have is a hex editor that does not know about the FAT file system. First step is to view the boot sector, which is located in sector 0 of the file system. There are six reserved sectors, two FATs, and each FAT is 249 sectors. Each cluster is 32 sectors, and there are 512 directory entries in the root directory.

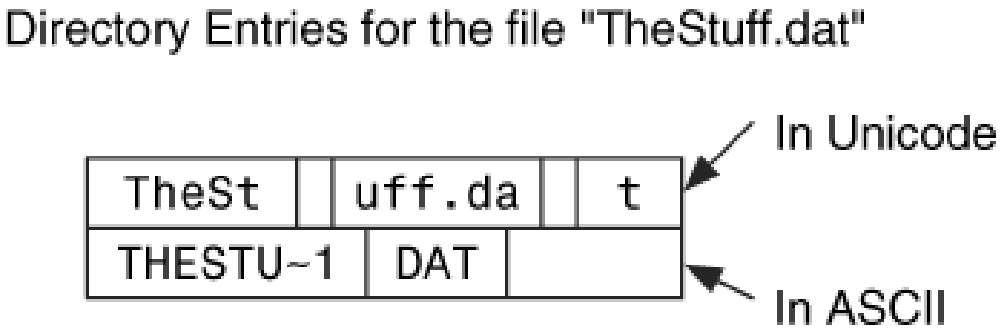


ANALYSIS SCENARIO: METADATA CATEGORY

- **File System Creation Date**: Encountered a FAT file system with only a few files and directories in it. This could be because the file system was not used much, because it was recently formatted, or because there is data being hidden from the investigator.
  - Look at the times of the directory entry with the volume label attribute set in the root directory.
  - Our analysis tool lists the directory entry as a normal file, and we see that it was formatted two weeks before the disk was acquired.
  - There could still be hidden data and data from the previous file system
- **Searching for Deleted Directories**:
  - extract the unallocated space using **dls** in TSK
  - search for the signature associated with the . directory entry that starts a directory i.e., 4-byte hex value 0x2e202020 which corresponds to the ASCII values for ". " using **sigfind**

ANALYSIS SCENARIO: FILENAME CATEGORY

- **File Name Searching** : search both allocated and deleted instances of a file named "TheStuff.dat"
  - If we do a keyword search of the file system for the name, we might find the content of files that reference our file.
  - won't find the file itself because the name is stored in an LFN entry and the SFN entry has a modified version of the name.
  - either search for "THESTUF~1DAT" in ASCII or
  - If we want to find the long version of the name, split the name up into chunks in Unicode.



ANALYSIS SCENARIO: FILENAME CATEGORY

- **Directory Entry Ordering:** found a directory full of contraband graphic images, and determine whether they were downloaded individually or copied from a CD? Directory entries in the directory are in alphabetical order that is strange. There are also no unallocated entries in the directory.
- Hypotheses:
  - It was copied or moved to this location, and the new entries were created in alphabetical order.
  - It was created by opening a ZIP file, and the unzip program created them in alphabetical order.
  - User downloaded each of these files in alphabetical order because that is the way they were listed on the remote server.

HYPOTHESIS 1

- Files that are the result of a copy will have a created time that is more recent than its last written time. In this case, all the files have a created time that is equal to the last written time, so copying does not seem likely if we can trust the temporal data.
- To test if the directory was moved, we create a directory with the same names as those we are investigating and create them in a non-alphabetical order.
- To test the moving hypothesis, we set the sorting option in the file manager to sort by name and then move the files by dragging and dropping.
- results in a directory with entries that are almost in alphabetical order, but not quite.
- If each file was moved individually in the sorted order, the new directory is sorted.

HYPOTHESIS 2

- Files are extracted from the ZIP file in the order that they were added to the ZIP file.
- If they were added in alphabetical order, they would be created in that order.
- resulting files have the same created and written times and they are equal to the original last written time of the file.

HYPOTHESIS 3

- this could occur, but the time values raise doubts about this scenario.
- Created and written times of the different files are days and sometimes months apart.
- Therefore, the user would have had to download these files in the sorted order over the course of months.
- This could be a reasonable situation if the files were released on a periodic basis and the naming convention of the files was based on when they were released.

Last two hypotheses are the most likely. For example, we can search for ZIP files or evidence of long-term downloading of these files to confirm the case.